

Extensiones de Machine Learning.

Práctica 2: Aprendizaje en Entornos Complejos.

José María Hernández Nieto

Juan de Dios Rodríguez Garrido

Javier García Serrano

{josemaria.hernandezn, juandios.rodriguezg, j.garciaserrano1}@um.es

9 de marzo de 2025

Resumen

Este documento tiene el propósito de documentar las actividades descritas en la Práctica 2: Aprendizaje en Entornos Complejos de la asignatura de Extensiones de Machine Learning del Máster en Inteligencia Artificial de la Universidad de Murcia. A través de las distintas secciones propuestas, introduciremos el problema propuesto en esta práctica, teniendo en cuenta la literatura relacionada. Describiremos el desarrollo seguido para la realización de la actividad, además de los algoritmos utilizados y resultados obtenidos con su correspondiente discusión y conclusión.

La parte experimental de esta práctica puede encontrarse en el siguiente repositorio: https://github.com/JoseHernandez11/RL_JMHJDRJG [1].

1. Introducción

El problema de exploración y explotación en entornos de toma de decisiones es un desafío fundamental en el campo del aprendizaje por refuerzo. Tradicionalmente, el problema del bandido de k-brazos se ha utilizado para ilustrar este dilema en entornos estáticos. Sin embargo, en escenarios más complejos, donde las decisiones se toman en contextos cambiantes, surgen estrategias más avanzadas, como los bandidos contextuales o los procesos de decisión de Markov con recompensas estocásticas [2].

La resolución de estos problemas se basa en las ecuaciones de Bellman, que pueden abordarse mediante cálculo matricial o iterativo, asumiendo que se dispone de un conocimiento completo del entorno. Sin embargo, en la mayoría de los casos reales, no se cuenta con información previa sobre la dinámica del sistema ni sobre las recompensas esperadas, lo que requiere estrategias que permitan al agente aprender a partir de su propia experiencia. Esto ha dado lugar a una serie de métodos basados en la interacción con el entorno, en los que el agente observa el estado, ejecuta una acción según una política, recibe una recompensa y ajusta su estrategia en consecuencia.

Las técnicas de aprendizaje por refuerzo se pueden clasificar en dos enfoques principales: los métodos de Monte Carlo y los de Diferencias Temporales [3]. Mientras que los primeros esperan a la finalización de un episodio para actualizar la política de toma de decisiones, los segundos ajustan la política de manera incremental a lo largo de la ejecución. Además,

cuando el espacio de estados y acciones es demasiado grande, se requiere el uso de funciones de aproximación mediante técnicas como el descenso del gradiente o el uso de redes neuronales, lo que da lugar a enfoques avanzados como el Deep Q-Learning [4].

El presente documento tiene como objetivo documentar el estudio y la experimentación con distintas técnicas de aprendizaje por refuerzo dentro del entorno de simulación Gymnasium [5]. Se analizarán y compararán diferentes estrategias, incluyendo métodos tabulares como Monte Carlo y SARSA, así como enfoques con aproximaciones como SARSA semi-gradiente y Deep Q-Learning. Para ello, se implementarán agentes de aprendizaje que serán evaluados en distintos entornos de Gymnasium, con el fin de identificar situaciones en las que sea necesario recurrir a métodos de aproximación. A lo largo del estudio, se modificará y adaptará código base para desarrollar implementaciones correctas de los algoritmos, asegurando una comprensión sólida de su funcionamiento y aplicabilidad en distintos escenarios.

El contenido del artículo se presenta a continuación siguiendo la siguiente estructura lógica: en la *Sección 2* se definirá formalmente el problema abordado, realizando una breve revisión bibliográfica y detallando los métodos utilizados durante el trabajo; en la *Sección 3*, se motivarán y explicarán detalladamente los algoritmos implementados en los estudios realizados; y posteriormente, se presentarán los resultados obtenidos de la fase de experimentación en la *Sección 4*, realizando además una discusión sobre el desempeño de estos; finalmente, en la *Sección 5*, concluiremos el artículo resumiendo los descubrimientos principales y su impacto, detallando posibles limitaciones y mejoras para futuros estudios.

2. Desarrollo

El problema central abordado en esta investigación es la toma de decisiones secuenciales en entornos inciertos mediante técnicas de aprendizaje por refuerzo. Específicamente, nos enfocamos en métodos tabulares como Monte Carlo (on-policy y off-policy), SARSA y Q-learning, así como en enfoques basados en aproximaciones funcionales, tales como SARSA semi-gradiente y Deep Q-Learning (DQN). Estos métodos permiten a un agente aprender una política π^* óptima a partir de la interacción con su entorno, sin conocer previamente el modelo subyacente. Los escenarios donde estos algoritmos son aplicables incluyen problemas de control en sistemas dinámi-

cos, optimización de estrategias en juegos, gestión de inventarios, y automatización de procesos industriales. En el presente trabajo, el foco está en la evaluación empírica de estos métodos en entornos concretos de Gymnasium, con el objetivo de analizar sus fortalezas y debilidades.

El aprendizaje por refuerzo ha sido ampliamente estudiado desde la formulación de los procesos de decisión de Markov (MDP) (Puterman 1994) [2]. Sutton y Barto (2018) [3] proporcionan una de las referencias más completas sobre el aprendizaje por refuerzo, describiendo tanto métodos tabulares como aquellos basados en aproximaciones funcionales. Los métodos tabulares incluyen:

- ▶ **Monte Carlo (MC)**: Se trata de un aprendizaje basado en episodios donde se estiman los valores de estado-acción promediando las recompensas obtenidas a lo largo de episodios completos (Sutton & Barto, 2018) [3]. Podemos diferenciar dos variantes:
 - **On-policy**: Se utiliza la misma política con la que explora para actualizar sus valores de estado-acción. Es útil cuando se busca aprender y mejorar continuamente la política óptima a partir de la misma fuente de exploración. Se suele aplicar en problemas donde la convergencia a una política estable es más importante que la exploración agresiva.
 - **Off-policy**: En ella se permite aprender una política óptima mientras explora con una diferente (e.g., usando la técnica de importancia ponderada). Es útil en entornos donde la exploración debe ser más agresiva o donde se quiere reutilizar datos de exploraciones previas, como en control de tráfico o robótica autónoma.
- ▶ **SARSA**: Este algoritmo basado en diferencias temporales (TD) actualiza los valores $Q(s, a)$ utilizando la transición (s, a, r, s', a') . Es un método on-policy, lo que significa que actualiza su política usando la misma estrategia de exploración con la que recopila datos (Rummery & Niranjan, 1994) [6]. Se suele usar en escenarios donde se requiere un equilibrio entre exploración y explotación controlada, como en sistemas de navegación autónoma o juegos con estrategia adaptable.
- ▶ **Q-learning**: Se trata de un método off-policy que estima los valores $Q(s, a)$ maximizando sobre las acciones futuras, permitiendo un aprendizaje más eficiente. Se destaca en problemas donde la evaluación de múltiples estrategias es crucial, como en el trading financiero o el control de inventarios (Watkins & Dayan, 1992) [7].

Cuando el espacio de estados y acciones es demasiado grande para una representación tabular, se requieren aproximaciones funcionales:

- ▶ **SARSA semi-gradiente**: Usa funciones de aproximación lineales o redes neuronales poco profundas para estimar valores de acción (Sutton & Barto, 2018) [3]. Este método se suele aplicar en problemas de gran escala donde la representación tabular es inviable, como en la predicción de demanda en redes eléctricas o sistemas de recomendación.

- ▶ **Deep Q-Learning (DQN)**: Este último método fue introducido por Mnih et al. (2015) [4], y usa redes neuronales profundas para representar la función Q , permitiendo manejar problemas de alta dimensionalidad. Destaca en aplicaciones donde la percepción del entorno es compleja, como en visión por computadora aplicada a robótica y juegos de estrategia.

El trabajo aquí desarrollado se diferencia de otros estudios previos al proporcionar una comparación detallada de métodos tabulares y de aproximación en entornos específicos de Gymnasium, demostrando cómo la complejidad del entorno afecta la elección adecuada de algoritmo. Mientras que estudios anteriores han mostrado la eficacia de DQN en problemas como Atari, nuestro análisis enfatiza la transición desde métodos tabulares hacia aproximaciones funcionales en un contexto controlado y didáctico. Con este estudio, buscamos proporcionar un marco sólido para la selección de algoritmos de RL, facilitando la comprensión de sus ventajas y limitaciones en entornos específicos. En la fase de experimentación, diferenciaremos dos partes:

- A) Primero estudiaremos un código base [8] facilitado por el profesor de la asignatura y lo modificaremos para que construya un agente de acuerdo a las indicaciones de Gymnasium para que aprenda siguiendo el algoritmo explicado en el notebook. Además, se construirá la gráfica de $f(t) = \text{len}(\text{episodio}_t)$.
- B) A continuación, desarrollaremos un estudio comparativo entre las distintas técnicas presentadas anteriormente, seleccionando dos entornos de Gymnasium [5], poniendo en manifiesto uno de ellos la necesidad de métodos aproximados.

3. Algoritmos

En este apartado pasaremos a la explicación de los algoritmos utilizados, incluyendo su correspondiente pseudocódigo cuando corresponda. Para ello, seguiremos la separación introducida en la última parte de la sección anterior. Esta separación, se mantendrá además presente a lo largo del artículo.

3.1. Algoritmos Parte A

Esta parte del trabajo aplica aprendizaje por refuerzo en los entornos Frozen Lake 4x4 y 8x8 de Gymnasium [9], utilizando el método Monte Carlo *on-policy*. Para ello, se ha desarrollado un agente modular, **MCAgent**, según la documentación aportada en la librería mencionada, que integra la actualización incremental de la función de valor Q y la política ϵ -greedy (ya definida en el código base).

Para la creación del agente Monte Carlo, se parte de un documento base en el que se incluye un algoritmo de entrenamiento inspirado en dicho método para el mecanismo de actualización de la tabla de valores, aunque no incorpora la funcionalidad que debería en su totalidad. Para lograr un algoritmo con MC puro, se ha modificado el almacenamiento de recompensas, se ha eliminado la variable acumulativa global, y se modificó también el cálculo del retorno G para que se realice cada paso, la actualización de la función Q y la actualización

de estadísticas.

Dicho algoritmo final se engloba dentro de una clase que constituye el agente del algoritmo de RL. La clase **MCAgent** encapsula:

- ▶ La tabla Q y la matriz de visitas, que se actualizan conforme al aprendizaje.
- ▶ El método **choose_action** para seleccionar acciones según la política ϵ -greedy.
- ▶ El método **train**, que itera en un número determinado de episodios, acumulando recompensas y actualizando Q .

Por otro lado, la política ϵ -greedy controla la probabilidad de realizar exploración en las distintas iteraciones ejecutadas a lo largo del tiempo. Concretamente, reduce la probabilidad de explorar nuevos caminos a medida que se avanza en el entrenamiento, permitiendo una mayor exploración al inicio y explotando los mejores resultados al final. La probabilidad de exploración en función del tiempo, utilizando la política ϵ -greedy, se puede definir como:

$$\epsilon(t) = \epsilon_{\min} + (\epsilon_0 - \epsilon_{\min}) e^{-\lambda t}, \quad (1)$$

donde:

- ▶ ϵ_0 es la probabilidad de exploración inicial,
- ▶ $\epsilon_{\min}$ es la probabilidad mínima de exploración, y
- ▶ λ es la tasa de decaimiento.

Además, se ha añadido un criterio de corte anticipado: una vez identificado un episodio exitoso (con recompensa 1), se guarda el número mínimo de pasos del camino hallado y, en episodios futuros, si se alcanza dicha longitud sin éxito, se interrumpe el episodio para concentrar la exploración en trayectorias óptimas.

El pseudocódigo completo del agente con las políticas mencionadas se muestra en el Algoritmo 1.

3.2. Algoritmos Parte B

Para la ejecución de la segunda parte experimental se implementaron los algoritmos descritos en la Sección 2, utilizando el conocimiento adquirido en clase y código base en la red de forma puntual. Comentamos a continuación su utilidad principal y parámetros utilizados:

- ▶ El método de Monte Carlo (MC) estima los valores de estado-acción a partir de episodios completos. En la modalidad On-Policy (Algoritmo 2), el agente sigue la misma política para explorar y actualizar valores, mientras que en Off-Policy (Algoritmo 3) usa una política distinta para explorar y otra para aprender, en nuestro caso utilizamos una política exploratoria uniforme. A continuación mostramos los parámetros utilizados para la implementación del algoritmo:
 - $num_{episodes} = 10^5$: Número total de episodios.
 - $gamma = 1,0$: Factor de descuento que pondera recompensas futuras.
 - $epsilon = 0,3$: Parámetro para la política epsilon-greedy en On-Policy.

- ▶ SARSA (Algoritmo 4) y Q-Learning (Algoritmo 5) son métodos de diferencias temporales (TD) que actualizan valores de estado-acción en cada paso del episodio. Para ambos algoritmos utilizamos los siguientes parámetros:
 - $total_{episodes} = 10^5$: Número de episodios.
 - $alpha = 0,1$: Tasa de aprendizaje.
 - $gamma = 0,99$: Factor de descuento.
 - $epsilon = 0,1$: Exploración epsilon-greedy.

- ▶ Cuando el espacio de estados es muy grande, las técnicas tabulares se vuelven inviables. SARSA Semi-Gradient (Algoritmo 6) usa una función lineal para representar Q , mientras que Deep Q-Learning (DQN, Algoritmo 7) emplea redes neuronales profundas, en nuestro caso utilizamos una red con 3 capas y 64 neuronas en la Hidden Layer para el primer entorno y otra con 2 capas y 32 neuronas para el segundo entorno.

4. Evaluación y experimentos

En esta sección presentaremos los resultados obtenidos en cada parte anteriormente propuesta, incluyendo además una discusión sobre estos.

4.1. Evaluación Parte A

A partir de funciones propias de Gymnasium [5] se inicializa el entorno Frozen Lake. De igual forma, para los dos entornos estudiados, se entrenó al agente durante 50 000 episodios, registrándose la evolución de la proporción de aciertos y la longitud de los episodios.

La longitud de los episodios es un indicativo de aprendizaje dado que, con la política empleada, al haber mayor explotación con el tiempo, si el aprendizaje es adecuado se debe de observar una convergencia en la longitud de los episodios. Esto se debe a que el agente encuentra cada vez caminos más cortos a medida que explora nuevas acciones.

La evaluación se realizó utilizando el IDE VSCode con un kernel de Python 3.12.0.

Como se puede observar en la Figura 1c, el número de episodios desciende rápidamente en el momento en el que se encuentra un camino exitoso, y todos los intentos de exploración posteriores se truncan al número de pasos del mejor camino encontrado hasta el momento. Por esa razón se puede observar la apariencia de escalera descendente a medida que avanza el entrenamiento. El tiempo de entrenamiento fue de 24,6 s.

Como se puede observar en la Figura 1d, el entorno 8x8, al ser un escenario similar pero con mayor complejidad, se observaron tendencias similares, aunque se requiere un mayor número de episodios para la convergencia. El criterio de corte anticipado permitió centrar la búsqueda en trayectorias eficientes, reduciendo significativamente el tiempo de entrenamiento. La comparación entre ambos entornos evidencia la escalabilidad del método y destaca la eficacia del criterio de corte en trayectorias óptimas. El tiempo de entrenamiento fue de 70,9 s.

Las gráficas mostradas muestran una clara tendencia convergente tanto en proporción de aciertos como en longitud de episodios. La proporción de aciertos demuestra que el agente

aprende con el tiempo. Además, este fenómeno está relacionado con la política epsilon-greedy, que a medida que epsilon disminuye hay mayor explotación, dando como resultado que el agente encuentre la meta con éxito cada vez más frecuentemente.

En caso de, por ejemplo utilizar el algoritmo off-policy, es decir con epsilon fijo, aún cuando el agente fuese capaz de aprender, no se apreciaría la misma convergencia a 1 en la proporción de aciertos.

4.2. Evaluación Parte B

En esta parte mostraremos ahora los resultados principales encontrados a partir de la experimentación llevada a cabo en la segunda parte del desarrollo. La representación principal que hemos escogido para analizar la eficacia del aprendizaje de cada modelo es la evolución de la longitud de los episodios en función del número de episodio.

Sobre la primera tanda de modelos, los cuales fueron entrenados sobre el entorno Frozen Lake, podemos observar su desempeño en la Figura 2. En primer lugar, para el método de Monte Carlo (Figura 2a), podemos notar una gran diferencia entre las modalidades On-Policy y Off-Policy, obteniendo esta segunda una longitud de episodios en promedio mucho menor a la primera, cosa que indica un mejor desempeño a la hora de llevar a cabo el aprendizaje sobre el entorno utilizado en concreto. En cuanto los métodos de diferencias temporales, vemos un caso parecido reflejado en la Figura 2b, donde el método SARSA obtiene longitudes medias significativamente menores que Q-Learning, evidenciado su mejor desempeño. Por último, de la Figura 2c vemos nuevamente cómo esta vez SARSA semi-gradiente desempeña mejor que Deep Q-Learning, al obtener longitudes medias menores otra vez.

Cabe destacar además la aparente tendencia ascendente en las tres gráficas del aumento de la longitud promedio de los episodios. Este fenómeno se debe a la combinación de elegir una representación logarítmica del eje x con la de dejar fijo el ratio entre exploración y explotación. De esta forma, eventualmente se producirán episodios altamente exploratorios donde la longitud de este sea descontroladamente elevada, obteniendo así un pico en la gráfica. Cuando combinamos esto con el aumento de densidad de puntos de la curva conforme avanzamos en el eje, estos picos puntuales aparecen mucho más juntos y más repetidamente (sólo en la visualización), apantallando el descenso promedio de la longitud de los episodios donde se lleve satisfactoriamente a la meta.

Considerando ahora las simulaciones realizadas sobre el entorno Mountain Car [10], cuyos resultados se muestran en la Figura 3, podemos observar ciertas diferencias respecto al entorno anterior. Este entorno introduce un aspecto novedoso frente al anterior, ya que para Frozen Lake teníamos un espacio de estados y acciones discretos, mientras que en este caso el espacio de estados es continuo y de dos variables (posición y velocidad de un vehículo) manteniendo un espacio de acciones discreto.

En primer lugar, podemos notar como la longitud de los episodios es considerablemente mayor que para el primer entorno,

evidenciando una complejidad mayor para llegar a una solución válida. Es importante además mencionar que a diferencia con Frozen Lake, para el cual un episodio acaba sólo si se llega a la meta o se cae a un agujero en principio (se puede aplicar otras condiciones), aquí sólo acaba el episodio si el coche alcanza la cima correcta o alcanza el límite de pasos, por lo que en este caso es más que destacable el proceso de aprendizaje al obtener exclusivamente soluciones correctas con una longitud de episodio paulatinamente menor, presentándose igualmente picos puntuales producidos por episodios con un alto carácter explorador. Este efecto es apreciable tanto para los algoritmos de diferencias temporales (SARSA y Q-Learning), mostrados en la Figura 3a, como para los de control de aproximaciones (SARSA semi-gradiente y Deep Q-Learning), cuyos resultados encontramos en la Figura 3b.

Para este entorno decidimos omitir los métodos de Monte Carlo, ya que como la estimación de los valores acción-estado se basa en la recompensa total obtenida al final del episodio y en los instantes iniciales no se obtiene recompensa positiva, este algoritmo no es capaz de llevar a cabo el aprendizaje, consumiendo además un tiempo de cómputo demasiado extenso debido a la discretización del espacio de estados, lo que genera una tabla de acción-estados de un gran tamaño. Por ello mismo, otros métodos como SARSA, Q-Learning y Deep Q-Learning son mejores alternativas para abordar este tipo de problemas.

Concretamente, podemos ver como los métodos de diferencias temporales logran resolver el problema alrededor de la iteración número 100 a partir de las mil iteraciones. Para el caso de control de aproximaciones, el algoritmo DQN fue ejecutado sólo para mil episodios, ya que su alta demanda computacional para entrenar la red neuronal tomaba tiempos demasiado largos, habiendo alcanzado nuevamente una solución correcta en 100 pasos. Para el caso de SARSA semi-gradiente, logra bajar aún más de la longitud 100, siendo el método muestra una mayor capacidad de aprendizaje en este entorno.

5. Conclusiones

Los resultados obtenidos en la experimentación con Frozen Lake y Mountain Car, previamente discutidos en la Sección 4, evidencian en último lugar la importancia de seleccionar el algoritmo adecuado según la complejidad y características del entorno. Mientras que Monte Carlo mostró buenos resultados en Frozen Lake, su aplicación en Mountain Car fue inviable debido a la ausencia de recompensas intermedias. SARSA y Q-Learning lograron aprendizaje eficiente en ambos entornos, aunque SARSA semi-gradiente superó a Q-Learning en espacios de estado continuos. Deep Q-Learning, aunque potente, requirió un mayor tiempo de entrenamiento. Para terminar, podemos concluir que este estudio reafirma la necesidad de combinar métodos tabulares y aproximados según el problema a tratar, optimizando exploración, eficiencia y convergencia.

A. Pseudocódigos de Algoritmos de Aprendizaje por Refuerzo

En este apéndice se recopilan todos los pseudocódigos referenciados en el documento, los cuales corresponden con los algo-

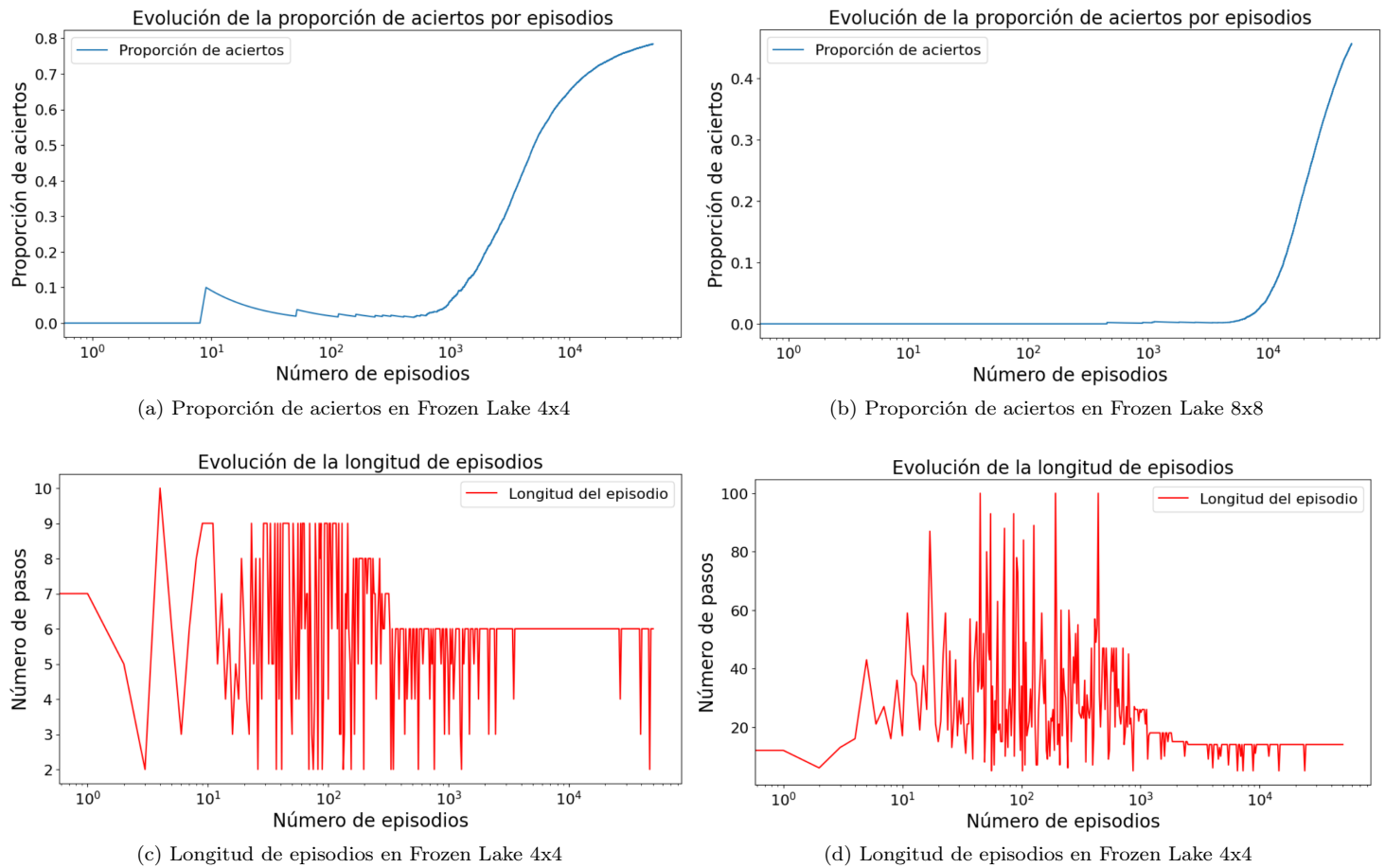


Figura 1: Resultados del entrenamiento en Frozen Lake.

ritmos implementados en cada parte correspondientemente.

En primer lugar, el Algoritmo 1 muestra el pseudocódigo que contiene la clase MC-Agent, utilizada en la evaluación parte A.

Por otro lado, el Algoritmo 2 se muestra el pseudocódigo del algoritmo de entrenamiento Monte Carlo On-Policy utilizado en la evaluación de la parte B.

En el Algoritmo 3 se muestra el pseudocódigo del algoritmo de entrenamiento Monte Carlo Off-Policy utilizado en la evaluación parte B.

En el Algoritmo 4 se muestra el pseudocódigo del algoritmo de entrenamiento SARSA utilizado en la evaluación de la parte B.

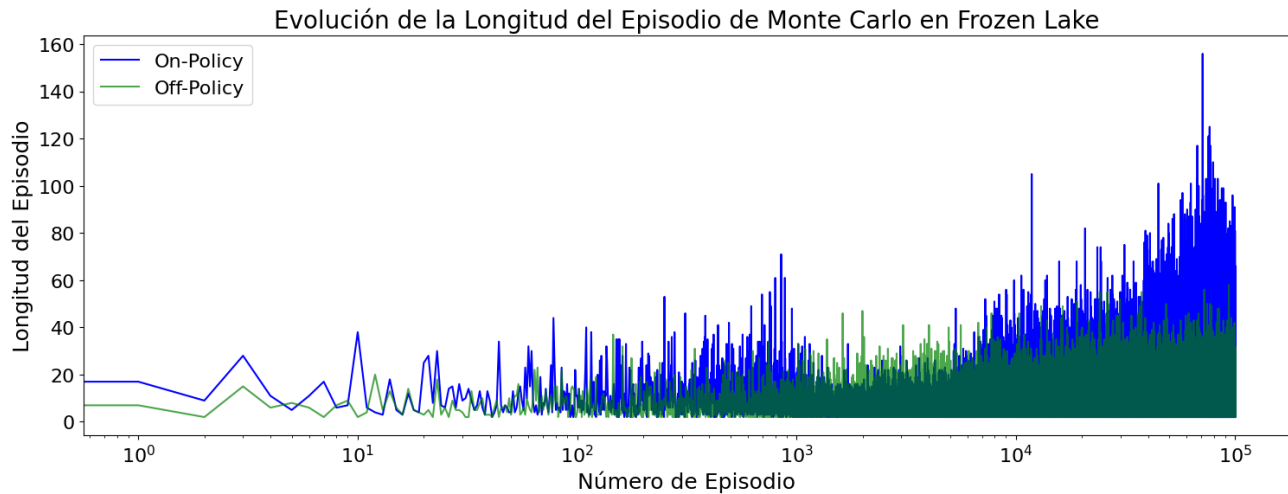
En el Algoritmo 5 se muestra el pseudocódigo del algoritmo de entrenamiento Q-Learning utilizado en la evaluación de la parte B.

En el Algoritmo 6 se muestra el pseudocódigo del algoritmo de entrenamiento SARSA Semi-Gradiente utilizado en la evaluación de la parte B.

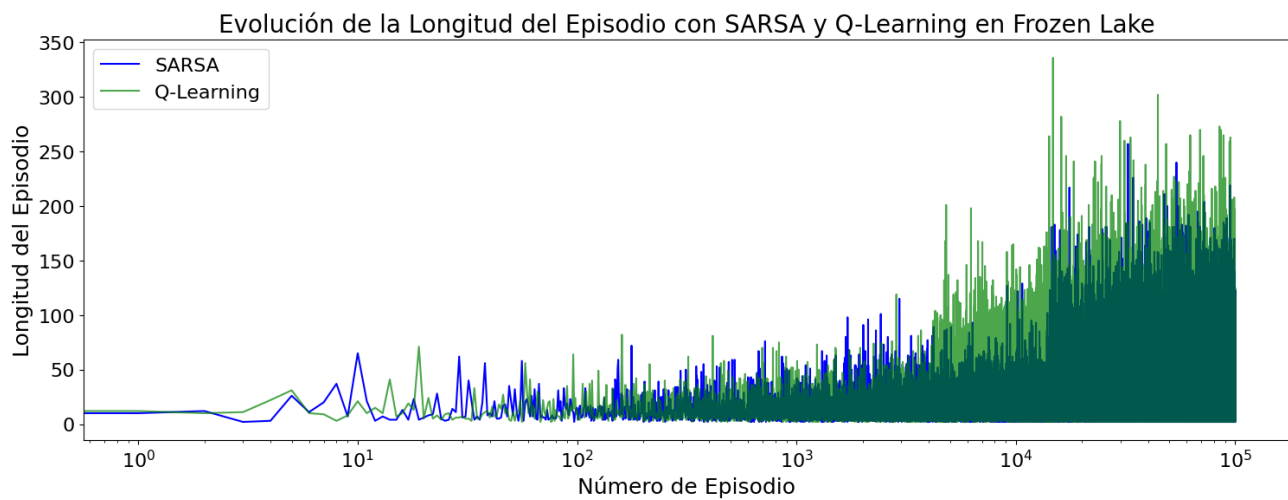
En el Algoritmo 7 se muestra el pseudocódigo del algoritmo de entrenamiento Deep Q-Learning utilizado en la evaluación de la parte B.

Referencias

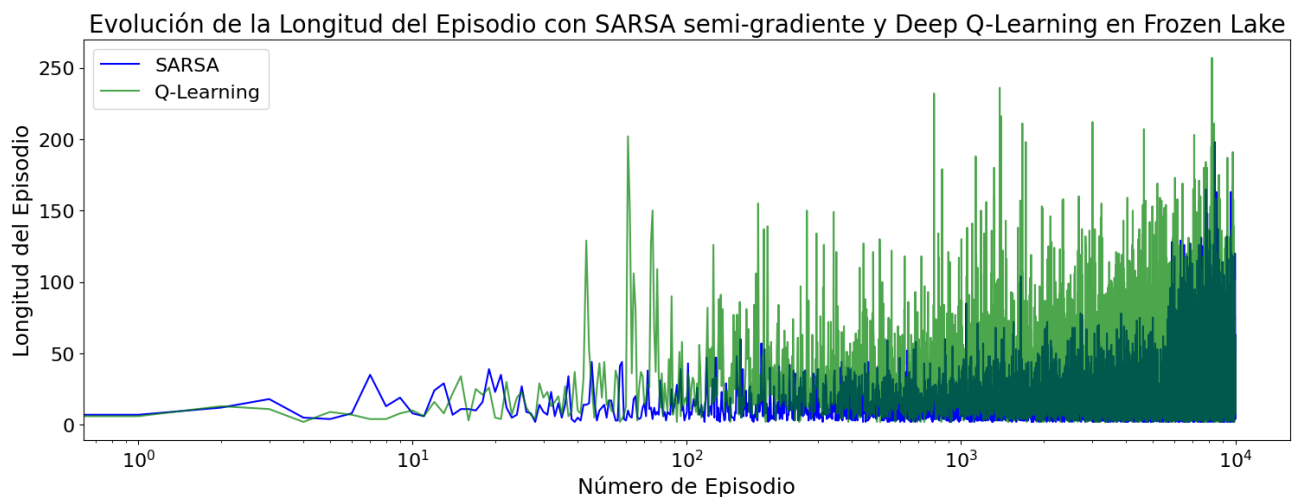
- [1] J. M. Hernández, J. de Dios Rodríguez, and J. García, "Repositorio github experimentos." https://github.com/JoseHernandez11/RL_JMHJDRJG, 2025. Accessed: March 2025.
- [2] M. L. Puterman, *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [3] R. S. Sutton, A. G. Barto, *et al.*, *Reinforcement learning: An introduction*, vol. 1. MIT press Cambridge, 1998.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] F. Foundation, "Gymnasium: A toolkit for developing and comparing reinforcement learning environments." <https://gymnasium.farama.org/>, 2023. Version 0.28.1, accessed March 2025.
- [6] G. A. Rummery and M. Niranjan, *On-line Q-learning using connectionist systems*, vol. 37. University of Cambridge, Department of Engineering Cambridge, UK, 1994.
- [7] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, pp. 279–292, 1992.



(a) Evolución de la longitud de episodios para el método de Monte Carlo On-policy y Off-policy.



(b) Evolución de la longitud de episodios para los métodos de SARSA y Q-Learning.



(c) Evolución de la longitud de episodios para los métodos de SARSA Semi-gradiente y Deep Q-Learning.

Figura 2: Resultados de las distintas técnicas consideradas sobre el entorno Frozen Lake de Gymnasium.

- [8] L. D. Hernández, “Código base parte a.” https://github.com/ldaniel-hm/eml_tabular/blob/main/MonteCarloTodasLasVisitas.ipynb, 2025. Accessed: March 2025.
- [9] “Frozen lake - gymnasium documentation.” https://gymnasium.farama.org/environments/toy_text/frozen_lake/.
- [10] “Mountain car - gymnasium documentation.” https://gymnasium.farama.org/environments/classic_control/mountain_car/.

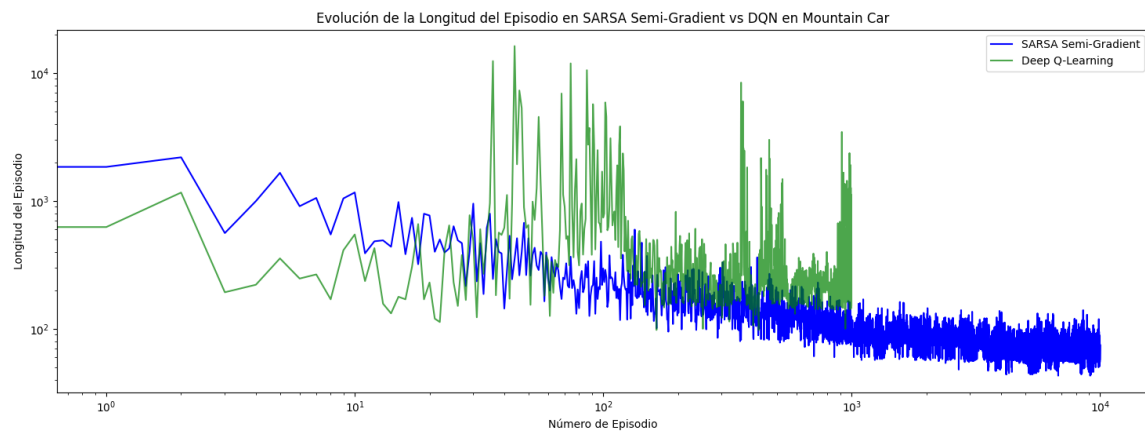
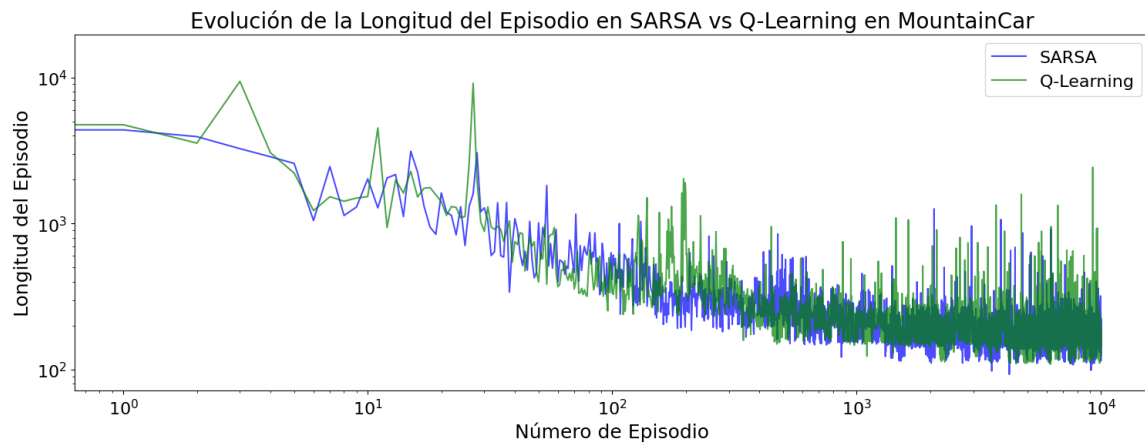


Figura 3: Resultados de las distintas técnicas consideradas sobre el entorno Mountain Car de Gymnasium.

Algorithm 1 Algoritmo de Entrenamiento de MCAgent

Require: Entorno env , número de episodios $num_episodes$, parámetro inicial ϵ y factor de descuento γ

- 1: Inicializar: $Q \leftarrow \mathbf{0}$, $n_visits \leftarrow \mathbf{0}$, $best_steps \leftarrow \text{None}$
- 2: **for** $t = 1$ **to** $num_episodes$ **do**
- 3: $(state, info) \leftarrow env.reset(seed = 100)$
- 4: $done \leftarrow \text{False}$
- 5: $episode \leftarrow \emptyset$ ▷ Lista de tuplas $(state, action, reward)$
- 6: $step_count \leftarrow 0$
- 7: **if** decay habilitado **then**
- 8: $\epsilon \leftarrow \min\left(1, 0, \frac{1000, 0}{t+1}\right)$
- 9: **end if**
- 10: **while** not $done$ **do**
- 11: $action \leftarrow \text{choose_action}(state)$ ▷ Utiliza política ϵ -greedy
- 12: $(new_state, reward, terminated, truncated, info) \leftarrow env.step(action)$
- 13: $step_count \leftarrow step_count + 1$
- 14: Agregar $(state, action, reward)$ a $episode$
- 15: $state \leftarrow new_state$
- 16: $done \leftarrow terminated \vee truncated$
- 17: **if** $best_steps \neq \text{None}$ and $step_count \geq best_steps$ **then**
- 18: $done \leftarrow \text{True}$ ▷ Corte anticipado
- 19: **end if**
- 20: **end while**
- 21: Registrar $step_count$ en $episode_lengths$
- 22: $G \leftarrow 0$
- 23: **for** cada tupla (s, a, r) en $episode$ recorrida en orden inverso **do**
- 24: $G \leftarrow r + \gamma \times G$
- 25: $n_visits[s, a] \leftarrow n_visits[s, a] + 1$
- 26: $\alpha \leftarrow \frac{1}{n_visits[s, a]}$
- 27: $Q[s, a] \leftarrow Q[s, a] + \alpha \times (G - Q[s, a])$
- 28: **end for**
- 29: Actualizar estadísticas acumuladas de recompensa usando el G del primer paso del episodio
- 30: **if** la recompensa $r = 1$ en el último paso del episodio **then**
- 31: **if** $best_steps$ es None **or** $step_count < best_steps$ **then**
- 32: $best_steps \leftarrow step_count$
- 33: **end if**
- 34: **end if**
- 35: (Opcional) Imprimir estadísticas de progreso
- 36: **end for**
- 37: **return** Q , estadísticas de recompensa, $episode_lengths$

Algorithm 2 Monte Carlo On-Policy

for cada episodio **do**

 Inicializar estado s

 Generar un episodio completo siguiendo una política π

for cada (s, a, r) en el episodio **do**

 Calcular retorno G acumulado hacia atrás

 Actualizar $Q(s, a)$ con el promedio de retornos

 Ajustar política usando epsilon-greedy

end for

end for

Algorithm 3 Monte Carlo Off-Policy

for cada episodio **do**

 Inicializar estado s

 Generar episodio siguiendo una política de exploración

 Inicializar peso $W = 1, 0$

for cada paso en el episodio, en orden inverso **do**

 Calcular retorno G

 Ajustar $Q(s, a)$ usando importancia muestral ponderada

 Si la acción no coincide con la política objetivo, detener actualización

 Actualizar W

end for

end for

Algorithm 4 SARSA

for cada episodio **do**

 Inicializar estado s

 Seleccionar acción a usando epsilon-greedy

while estado s no es terminal **do**

 Ejecutar a , observar recompensa r y nuevo estado s'

 Seleccionar nueva acción a' usando epsilon-greedy

 Actualizar $Q(s, a)$ con la ecuación de SARSA

$s \leftarrow s'$, $a \leftarrow a'$

end while

end for

Algorithm 5 Q-Learning

for cada episodio **do**

 Inicializar estado s

while estado s no es terminal **do**

 Seleccionar acción a usando epsilon-greedy

 Ejecutar a , observar recompensa r y nuevo estado s'

 Actualizar $Q(s, a)$ maximizando sobre acciones futuras en s'

$s \leftarrow s'$

end while

end for

Algorithm 6 SARSA Semi-Gradient

Inicializar pesos W

for cada episodio **do**

 Inicializar estado s , seleccionar acción a

while estado s no es terminal **do**

 Ejecutar a , observar recompensa r y nuevo estado s'

 Seleccionar nueva acción a' usando epsilon-greedy

 Calcular error TD y actualizar pesos W

$s \leftarrow s'$, $a \leftarrow a'$

end while

end for

Algorithm 7 Deep Q-Learning (DQN)

```
Inicializar red neuronal  $Q$  y memoria de repetición
for cada episodio do
  Inicializar estado  $s$ 
  while estado  $s$  no es terminal do
    Seleccionar acción usando epsilon-greedy
    Ejecutar acción y almacenar transición en memoria
    if hay suficiente memoria then
      Tomar minibatch de experiencias
      Calcular objetivo usando la red de destino
      Minimizar pérdida entre  $Q$  estimado y objetivo
    end if
    Actualizar parámetros de la red
  end while
end for
```
